

CS2601 Linear and Convex Optimization

Homework 4

Due: 2023.11.17

For problem 2., 3. and 4. of this assignment, we need CVXPY (<https://www.cvxpy.org/>).

For problem 5., 6. and 7., you will implement gradient descent with constant step size. First complete the function `gd_const_ss` in `gd.py`. You should submit your source code (.py or .ipynb files), all necessary figures, the outputs of your Python code, and your answers to the questions for 5., 6. and 7. Do NOT submit your figures in separate files. Your answers in any of the .py or .ipynb files will NOT be graded.

1. Determine if the following optimization problems are convex optimization problems and show your reasons.

(a).

$$\begin{aligned} \min_{x_1, x_2} \quad & 2x_1^2 - 2x_1x_2 + x_2^2 + x_1 + x_2 \\ \text{s.t.} \quad & (x_1 - x_2)^2 + 4x_1x_2 + e^{x_1+x_2} \leq 0 \\ & x_1 - 3x_2 = 0 \end{aligned}$$

(b).

$$\begin{aligned} \min_{x_1, x_2} \quad & x_1^2 + 3x_2^4 \\ \text{s.t.} \quad & x_1 e^{-(x_1+x_2)} \leq 0 \\ & x_1^2 - 2x_1x_2 + x_2^2 + x_1 + x_2 \leq 0 \\ & 5x_1^2 - 7x_2 = 0 \end{aligned}$$

2. Consider the optimization problem

$$\begin{aligned} \min_{x_1, x_2} \quad & f(x_1, x_2) \\ \text{s.t.} \quad & 2x_1 + x_2 \geq 1 \\ & x_1 + 4x_2 \geq 1 \\ & x_1 \geq 0, x_2 \geq 0 \end{aligned}$$

Sketch the feasible set. For the objective functions in (a)-(c), first find the set of optimal solutions and the optimal value graphically, and then solve them using CVXPY. For the objective functions in (d) and (e), solve them using CVXPY. Show the outputs of CVXPY. Check the first example at <https://www.cvxpy.org/tutorial/intro/index.html>. You can use `cvxpy.maximum(x,y)` to get the maximum of two scalar variables x and y , or `cvxpy.max(x)` to get the maximum of the components of a vector variables x .

(a). $f(x_1, x_2) = x_1 + x_2$

(b). $f(x_1, x_2) = -x_1 - x_2$

(c). $f(x_1, x_2) = x_1$

(d). $f(x_1, x_2) = \max\{x_1, x_2\}$

(e). $f(x_1, x_2) = x_1^2 + 8x_2^2$

3. Consider the following optimization problem with $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$.

$$\begin{aligned} \min_{\mathbf{x}} \quad & \|\mathbf{Ax} - \mathbf{b}\|_1 \\ \text{s.t.} \quad & \|\mathbf{x}\|_\infty \leq 1 \end{aligned} \tag{1}$$

(a). Reformulate problem (1) as an LP.

(b). Solve the original problem in (1) using CVXPY for $m = 3$, $n = 2$,

$$\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & -3 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 5 \\ 8 \\ -5 \end{pmatrix}.$$

Show the output of CVXPY. To see how to use vectors and matrices in CVXPY, check the examples at <https://www.cvxpy.org/tutorial/intro/index.html#vectors-and-matrices>. The p -norm of a vector $\mathbf{x}$ is given by `cvxpy.norm(x, p)`. Use "inf" to get the ∞ -norm. Alternatively, you can use `cvxpy.norm2(x)`, `cvxpy.norm1(x)`, `cvxpy.norm_inf(x)` to get the 2-, 1-, and ∞ -norms, respectively.

(c). Solve the LP problem you find in (a) using CVXPY. Show the output of CVXPY. You can check the example here https://www.cvxpy.org/examples/basic/linear_program.html.

4. Let $\mathbf{w} \in \mathbb{R}^2$ and

$$\mathbf{X} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 3 \\ 2 \\ 2 \end{pmatrix}.$$

Solve problem (a) by hand and problems (b), (c) with CVXPY and show the outputs.

(a). Linear least squares regression

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

(b). Lasso

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

$$\text{s.t. } \|\mathbf{w}\|_1 \leq t$$

for $t = 1$ and $t = 10$. In each case, is the solution the same as that of (a)? Does the solution have zero components? (Note that there are numerical errors, which you should ignore when answering the questions.)

(c). Ridge regression

$$\begin{aligned} \min_{\mathbf{w}} \quad & \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 \\ \text{s.t.} \quad & \|\mathbf{w}\|_2^2 \leq t \end{aligned}$$

for $t = 1$ and $t = 100$. In each case, is the solution the same as that of (a)? Does the solution have zero components? (Again ignore the numerical errors.)

5. Consider the following quadratic program,

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} \tag{2}$$

where

$$\mathbf{Q} = \begin{pmatrix} \gamma & 0 \\ 0 & 1 \end{pmatrix}.$$

with $\gamma > 0$. In terms of coordinates,

$$f(x_1, x_2) = \frac{\gamma}{2} x_1^2 + \frac{1}{2} x_2^2.$$

We know the optimal solution is $\mathbf{x}^* = \mathbf{0}$ and $f^* = f(\mathbf{x}^*) = 0$.

(a). What is the smallest L such that f is L -smooth?

(b). Suppose $\gamma = 0.1$. Complete **p5.py** and run the gradient descent algorithm you implemented with $\mathbf{x}_0 = (1, 1)$ and step sizes 2.2, 1, 0.1, 0.01. (Note: for step size 2.2, you may want to limit the maximum number of iterations to a small number, say 10.) Does it converge in each case? When it does converge, how many iterations does it take? In each case, plot the 2D trajectory of the sequence $\mathbf{x}_k$ and the function values $f(\mathbf{x}_k)$. You can use the function provided in **utils.py** or write your own code. Note that $f(\mathbf{x}_k)$ is also the suboptimality gap since $f(\mathbf{x}^*) = 0$.

(c). For $\gamma = 1, 0.1, 0.01, 0.001$, run gradient descent with step size 1. How many iterations do you need in each case? How does the number of iterations change as γ decreases? You don't need to show the plots.

6. **Linear regression.** Consider the least squares problem in Problem 4(a), i.e.

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

where

$$\mathbf{X} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 3 \\ 2 \\ 2 \end{pmatrix}.$$

Use your implementation of gradient descent to solve this least squares problem. You may reuse the codes for Problem 5 by rewriting `f` and `fp` (you don't have to plot the traces as in Problem 5). Use any stepsize you find appropriate. Compare your solution with the solution you found in Problem 4, and the solution you find by solving the normal equation using `np.linalg.solve`. Do they agree with each other?

7. Logistic regression. In this problem, you will train a logistic regression classifier on a toy dataset given in `p7.py`. Data points have two features, so the i -th sample is $(\mathbf{x}_i, y_i) \in \mathbb{R}^2 \times \{+1, -1\}$. As in Problem 2 of Homework 1, we will append a constant 1 to each $\mathbf{x}_i$ to make it three dimensional, and absorb the bias term b into $\mathbf{w}$ by defining a new $\mathbf{w} = (w_1, w_2, b)^T \in \mathbb{R}^3$. The objective function to be minimized is

$$f(\mathbf{w}) = \sum_{i=1}^m \log(1 + e^{-y_i \mathbf{x}_i^T \mathbf{w}})$$

where m is the number of samples. Its gradient is,

$$\nabla f(\mathbf{w}) = - \sum_{i=1}^m [1 - \sigma(y_i \mathbf{x}_i^T \mathbf{w})] y_i \mathbf{x}_i$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function.

Find the optimal $\mathbf{w}^*$ using your implementation of gradient descent in `gd.py`. Report the classification accuracy of the classifier on the training dataset given by

$$\text{accuracy} = \frac{1}{m} \sum_{i=1}^m \mathbb{1}\{y_i = \hat{y}_i\}$$

where $\hat{y}_i = \text{sgn}(\langle \mathbf{w}^*, \mathbf{x}_i \rangle)$. (Note that the accuracy is not the objective function to be minimized, but $f(\mathbf{w})$ is.) You can follow the guidance of `p7.py` or write your own code. Use matrix operations to calculate the function values and gradients. You may need some of the following functions, `numpy.matmul`, `numpy.multiply`, `numpy.exp`, `numpy.reshape`.